how policy-aware agent architectures can establish safe boundaries, and how techniques like policy constraints and delegated authority can influence agent behavior.

Along the way, we will see how ACME incorporated authorization directly into the agent's execution loop and how policies written in Cedar became a crucial part of governing agent activity. AI agents may represent a new class of actors in software systems, but the principles of effective authorization architecture still apply. If anything, they are more important than ever.

18.1 Why authorization is the hard problem in agentic AI

In the weeks after her arrival at ACME, Trudy started reviewing the company’s early AI experiments. Engineers integrated language models with internal tools. Sales teams used agents to prepare account briefings. Operations teams tested agents that could diagnose infrastructure issues and trigger automated workflows.

Most of these systems followed a similar pattern. A language model was wrapped in an agent framework that allowed it to interact with enterprise systems using various tools. These tools provided functions such as retrieving documents, querying databases, and calling internal APIs.

The architecture worked well technically. Agents could reason about problems, select tools, and combine their outputs to achieve complex goals. But as Trudy spoke with the teams developing these systems, she noticed that most conversations focused on what agents could do. Very few discussed what agents should be allowed to do.

Traditional authorization models assume software behaves predictably. As Trudy observed, that assumption no longer holds for agentic systems. Agents plan their actions dynamically. Given a goal, an agent can explore multiple strategies, use different tools, and adapt its plan as it acquires new information. The sequence of actions may differ from one execution to another. This flexibility is what makes agents powerful, but it also makes